

Systems Analysis and Design

Chapter 7: Introduction to UML



How can you explain the effect of
polluted water or **global warming**
issue?



1.5 MILLION CHILDREN
DIE EVERY YEAR FROM
DRINKING POLLUTED WATER.

WWW.UNICEF.ORG

unicef 

1 picture's worth **1000** words



Learning Objectives

7.1 History of UML

7.2 Systems, Models and Views

7.3 Design Use cases

7.4 Design Class diagrams

7.5 Design Interaction diagrams

7.6 Design another types of diagrams

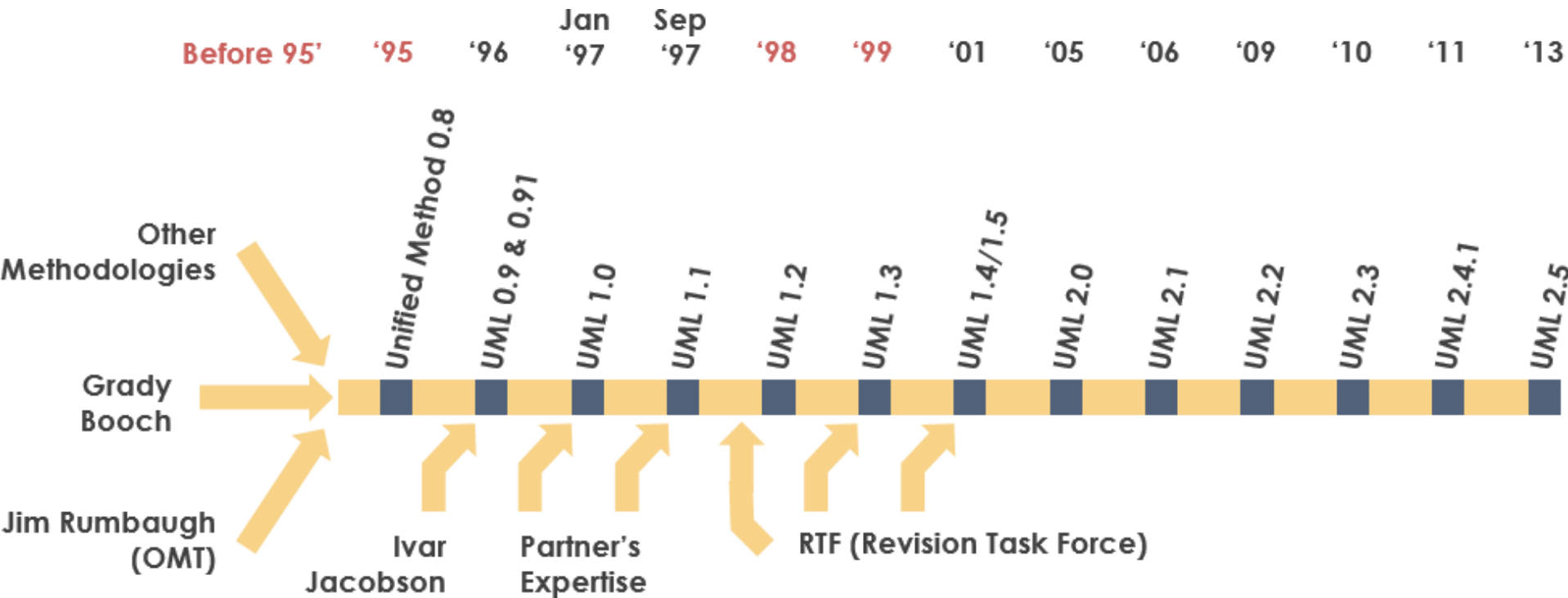
What is UML?

- Unified Modeling Language
 - OMG Standard, Object Management Group
 - Based on work from Booch, Rumbaugh, Jacobson
- UML is a modeling language to express and design documents, software
 - Particularly useful for OO design
 - Not a process, but some have been proposed using UML
 - Independent of implementation language

Why use UML?

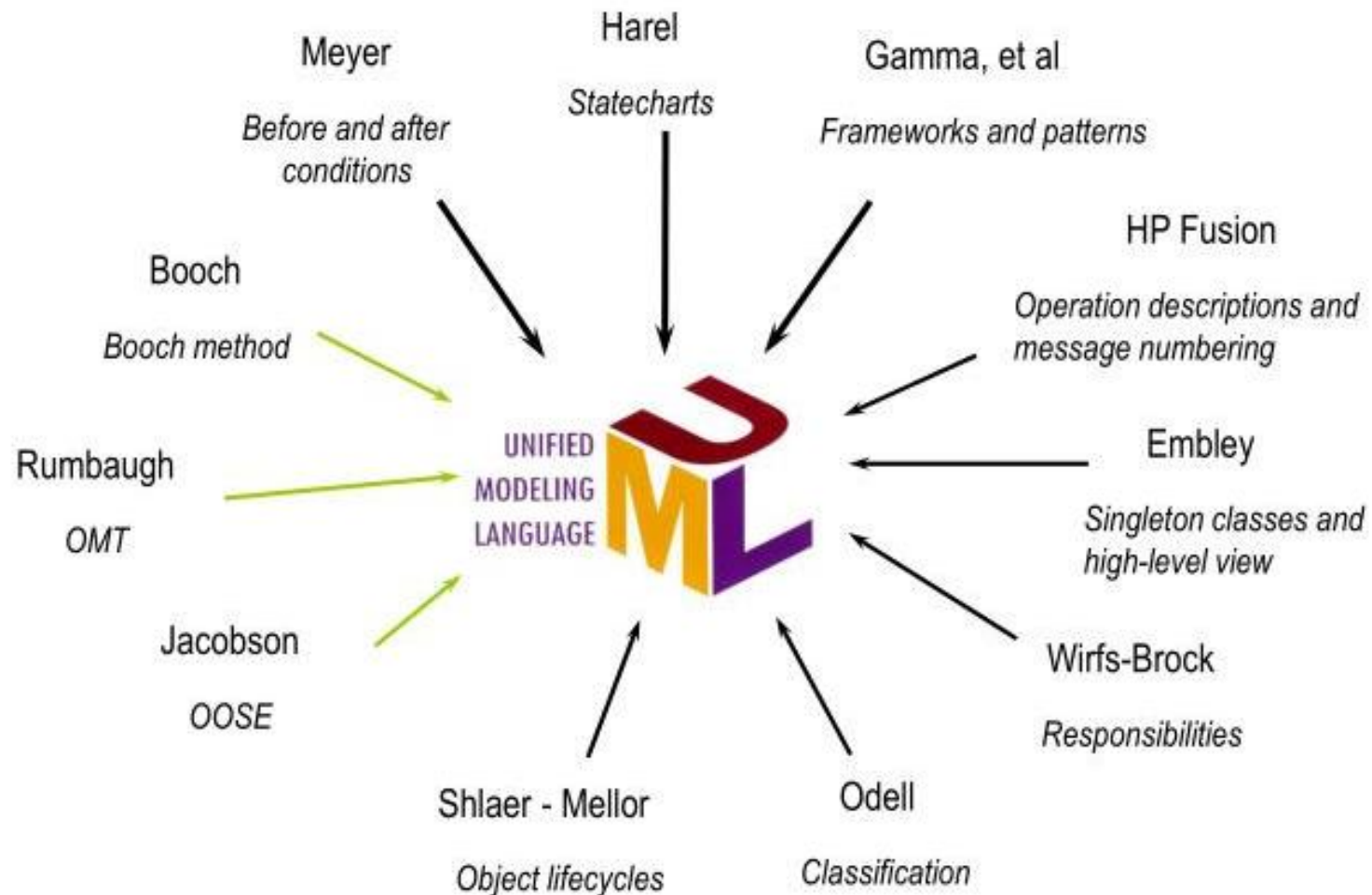
- Standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, business modeling and other non-software systems.
- The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.
- The UML is a very important part of developing object oriented software and the software development process.
- The UML uses mostly graphical notations to express the design of software projects.
- Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

History of UML



Before '95 - Fragmentation ► '95 - Unification ► '98 - Standardization ► '99 - Industrialization

Contributions to UML



UML Diagrams

- Structure diagram
 - Object diagram
 - Class diagram
 - Component diagram
 - Deployment diagram
 - Package diagram
 - Composite structure diagram (UML 2.2)
 - Profile diagram (UML 2.2)
- Behavior diagram
 - Interaction diagrams:
 - Sequence diagram
 - Collaboration diagram (communication diagram in UML 2.2)
 - Interaction overview diagram (UML 2.2)
 - Timing diagram (UML 2.2)
 - Use case diagram
 - Activity diagram
 - State diagram

Learning Objectives

7.1 History of UML

7.2 Systems, Models and Views

7.3 Design Use cases

7.4 Design Class diagrams

7.5 Design Interaction diagrams

7.6 Design another types of diagrams

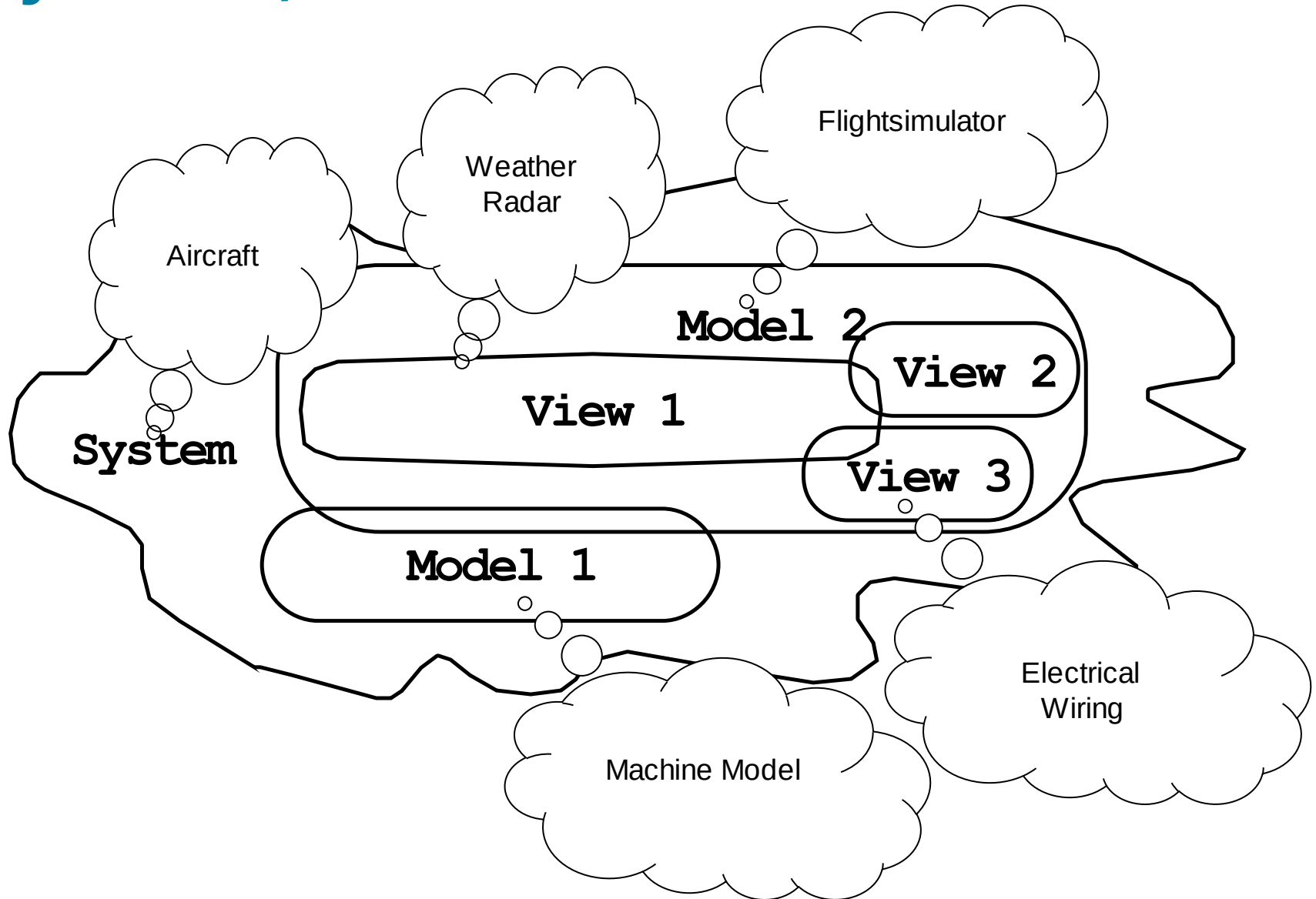
Systems, Models and Views

- A ***model*** is an abstraction describing a subset of a system
- A ***view*** depicts selected aspects of a model
- A ***notation*** is a set of graphical or textual rules for depicting views
- Views and models of a single system may overlap each other

Examples:

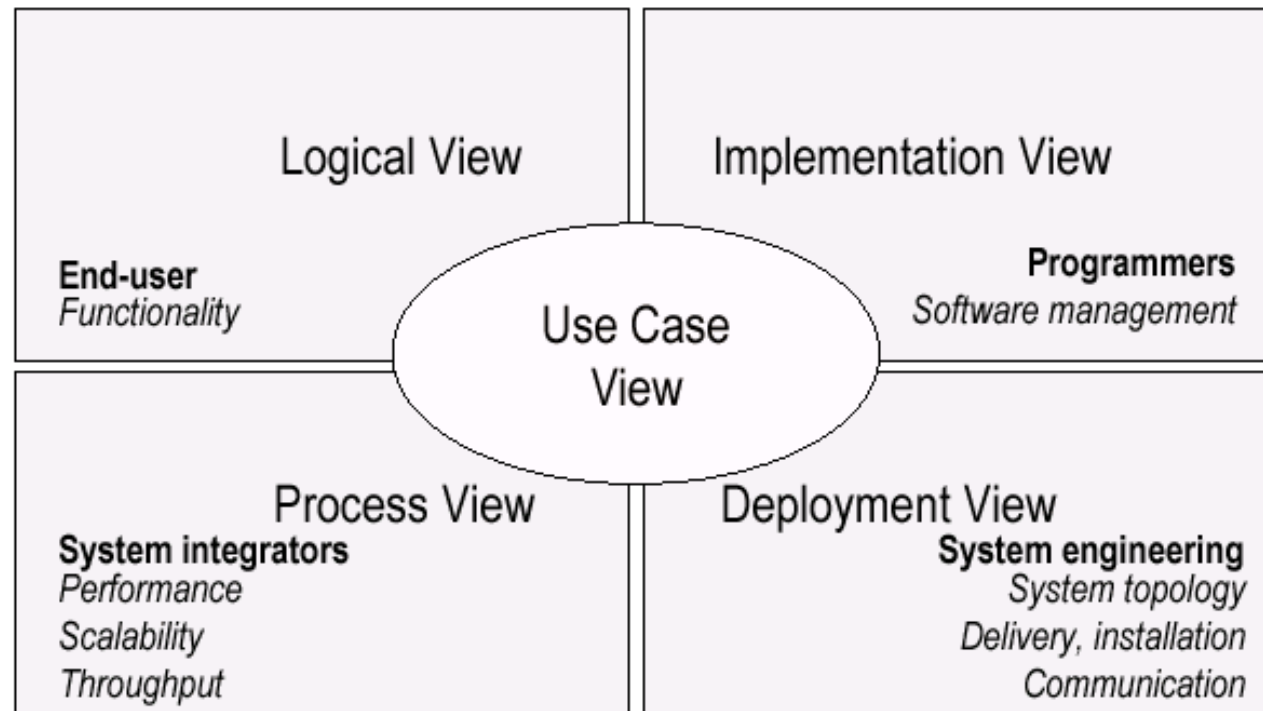
- System: Aircraft
- Models: Flight simulator, machine model
- Views: Radar, electrical wiring, fuel system

Systems, Models and Views

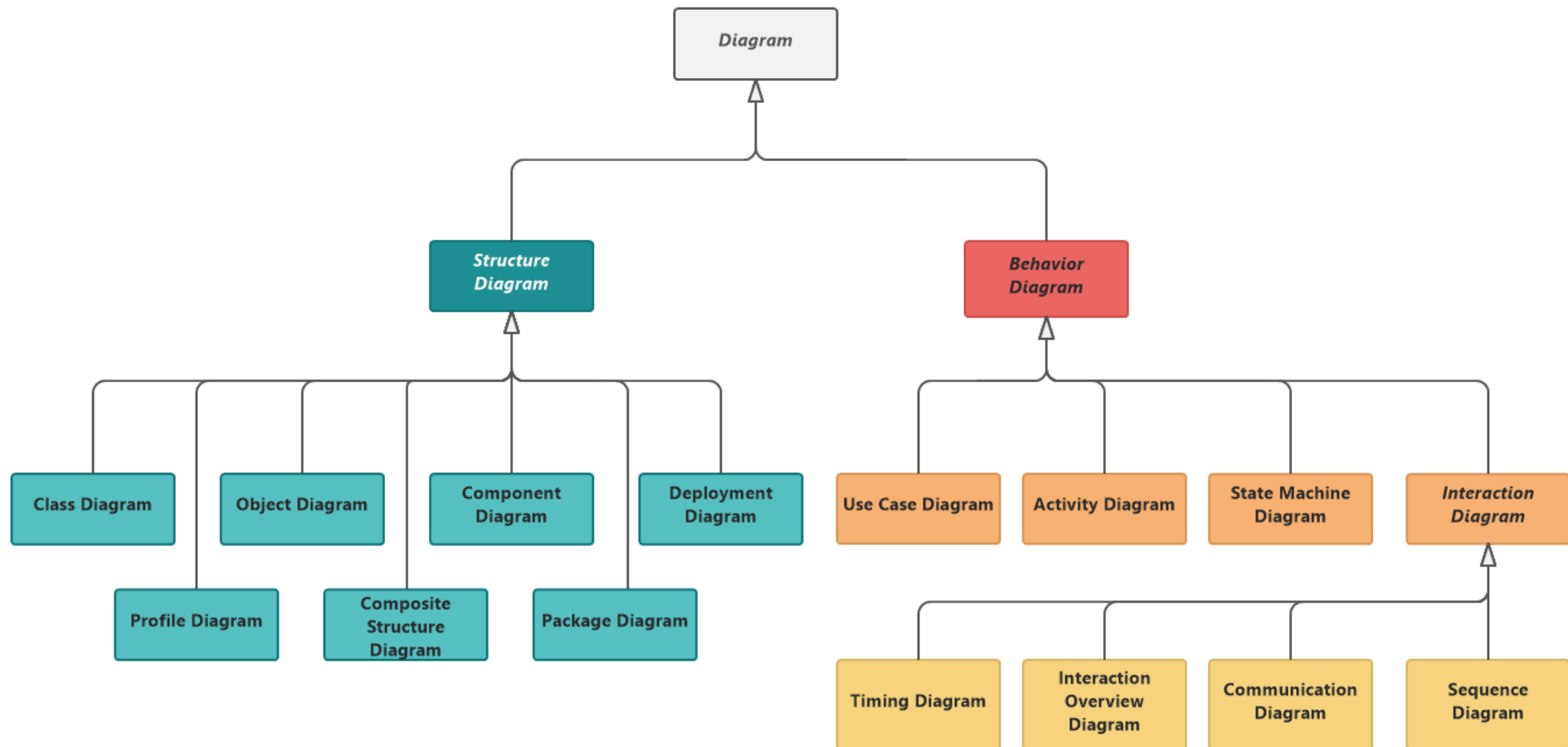


UML Models, Views, Diagrams

- UML is a multi-diagrammatic language, each diagram is a view into a model
 - Diagram presented from the aspect of a particular stakeholder
 - Provides a partial representation of the system
 - Is semantically consistent with other views
- Example views:



Type of Diagrams



How Many Views?

- Views should fit the context
 - Not all systems require all views
 - Single processor: drop deployment view
 - Single process: drop process view
 - Very small program: drop implementation view
- A system might need additional views
 - Data view, security view, ...
- You can model 80% of most problems by using about 20% UML diagrams
- We only cover the 20% here

Basic Modeling Steps

- Use Cases Model
 - Capture requirements
- Domain Model
 - Capture process, key classes
- Design Model
 - Capture details and behaviors of use cases and domain objects
 - Add classes that do the work and define the architecture

Learning Objectives

7.1 History of UML

7.2 Systems, Models and Views

7.3 Design Use cases

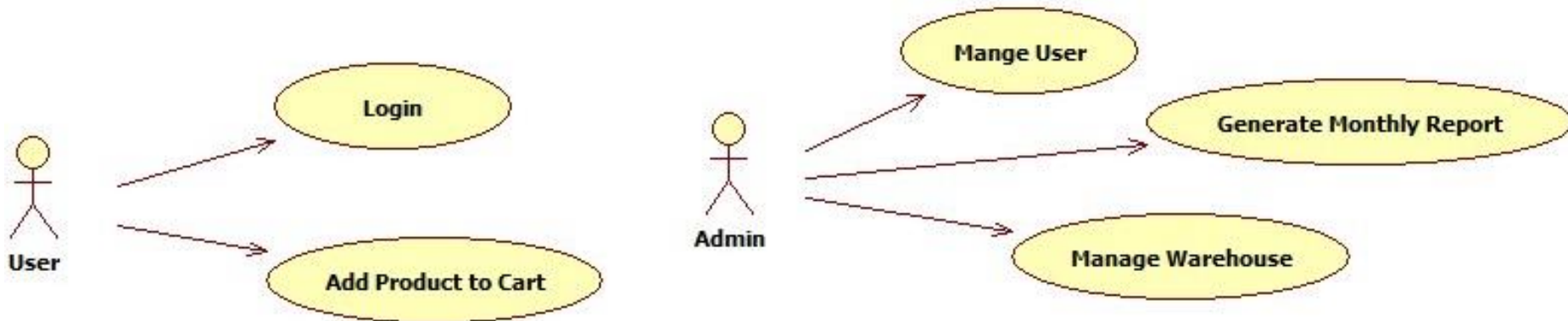
7.4 Design Class diagrams

7.5 Design Interaction diagrams

7.6 Design another types of diagrams

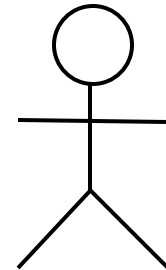
Use Case Diagrams

- Used during requirements elicitation to represent external behavior
 - **Actors** represent roles, that is, a type of user of the system
 - **Use cases** represent a sequence of interaction for a type of functionality; summary of scenarios
- The use case model is the set of all use cases. It is a complete description of the functionality of the system and its environment

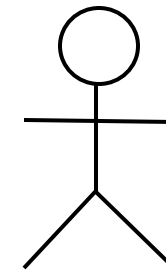


Actors

- An actor models an external entity which communicates with the system:
 - User
 - External system
 - Physical environment
- An actor has a unique name and an optional description.
- Examples:
 - Passenger: A person in the train
 - GPS satellite: Provides the system with GPS coordinates



User



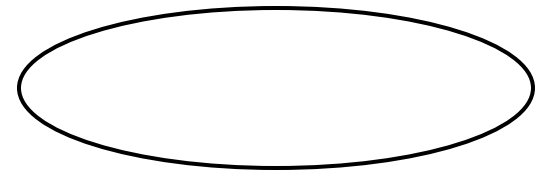
Admin

Use Case

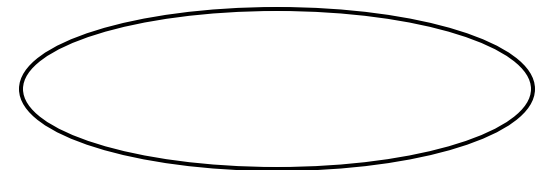
A use case represents a class functionality provided by the system as an event flow.

A use case consists of:

- Unique name
- Participating actors
- Entry conditions
- Flow of events
- Exit conditions
- Special requirements



Add Product to Cart



Manage User

Use Case Specification: Example

UC Name: Purchase ticket

Participating actor: Passenger

Preconditions:

- Passenger standing in front of ticket distributor.
- Passenger has sufficient money to purchase ticket.

Postconditions:

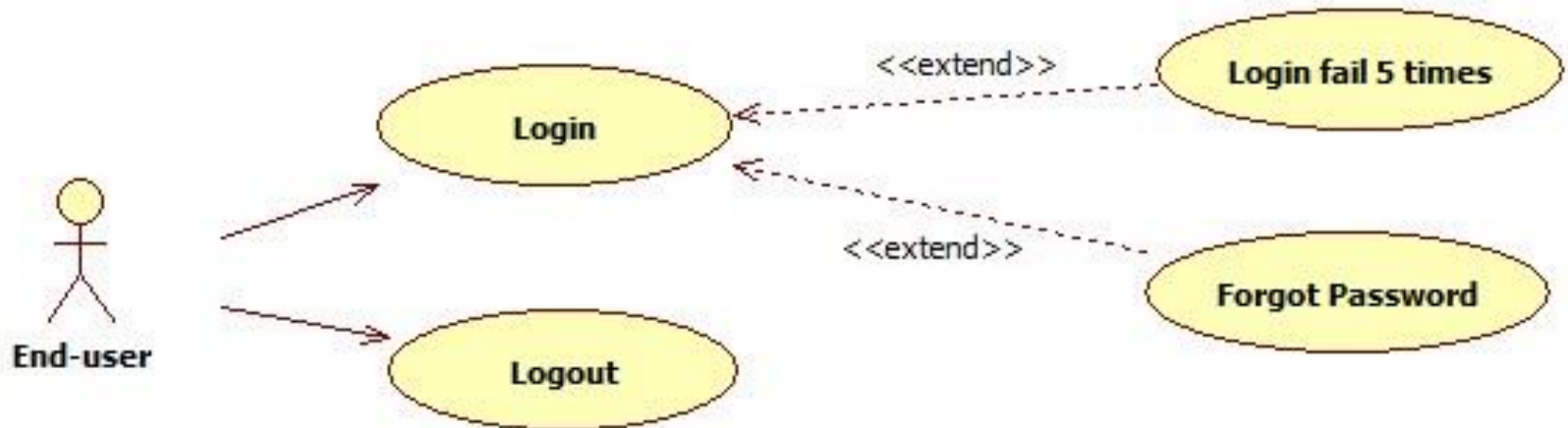
- Passenger has ticket.

Flow of Events:

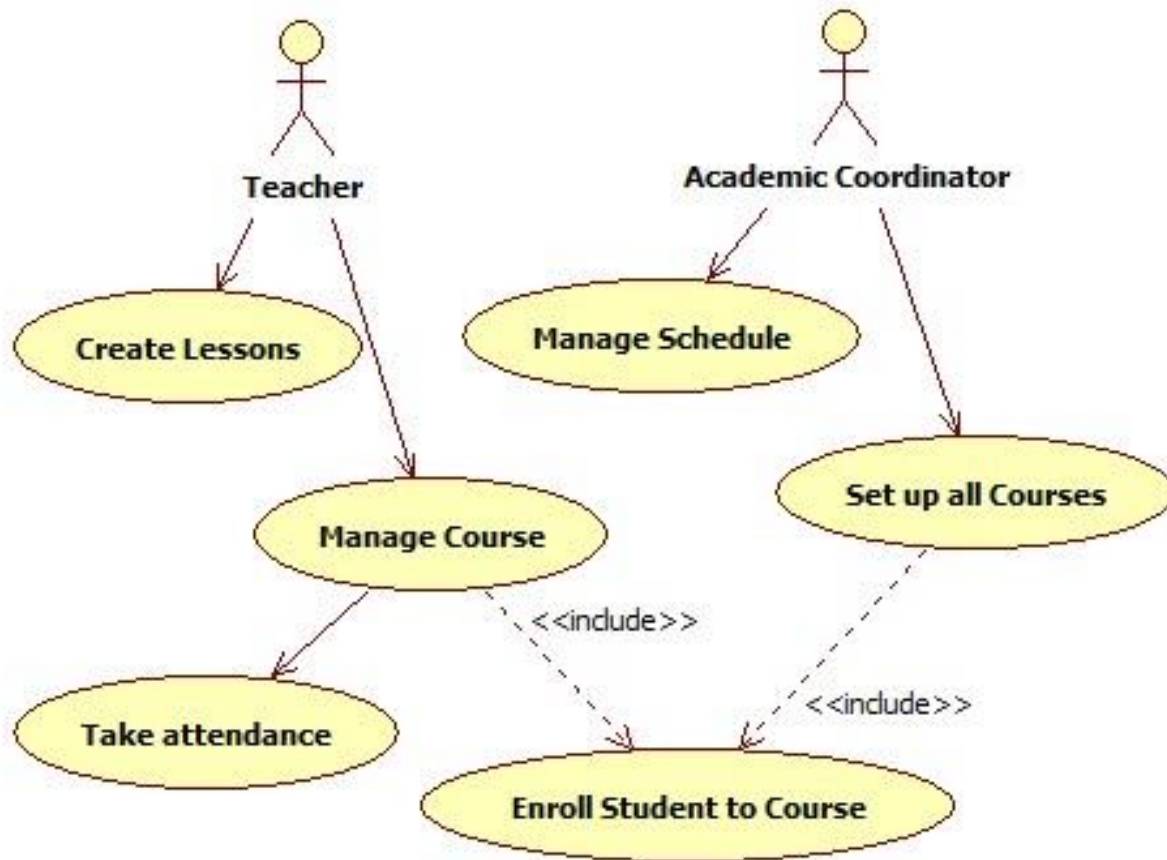
1. Passenger selects the number of zones to be traveled.
2. Distributor displays the amount due.
3. Passenger inserts money, of at least the amount due.
4. Distributor returns change.
5. Distributor issues ticket.

UC Association: *extend*

- <<extend>> relationships represent exceptional invoked cases
- The exceptional flows are factored out of the main event flow for clarity
- UCs representing exceptional flows can extend more than one UC



UC Association: *include*

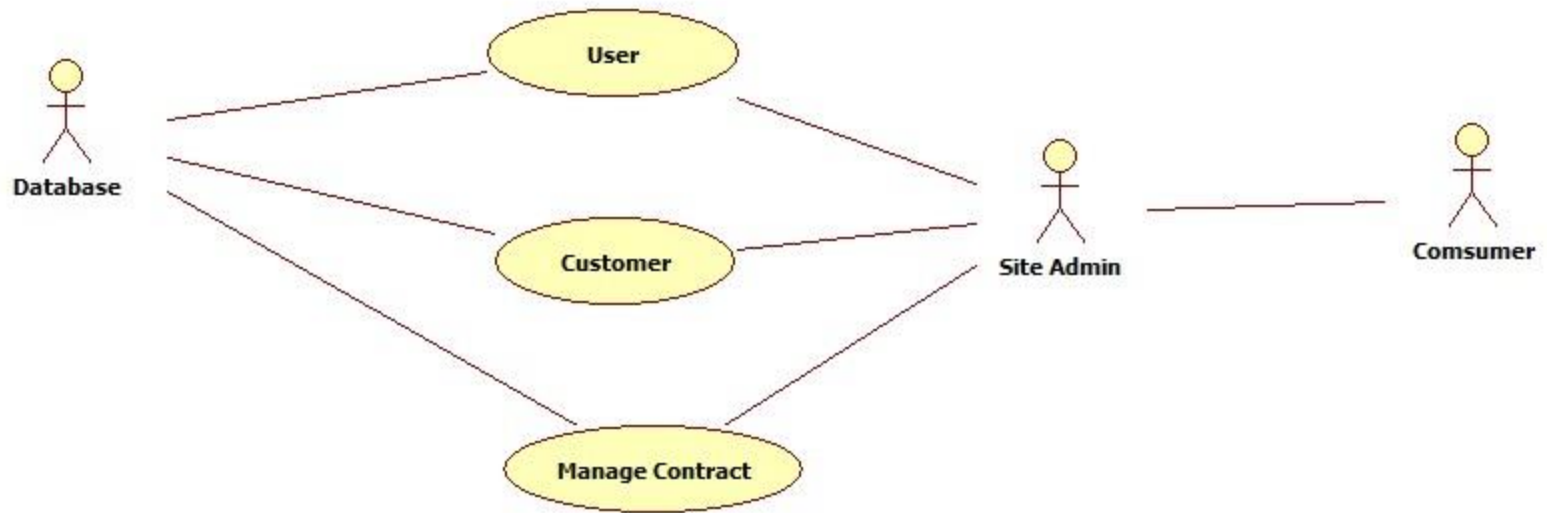


- <<include>> relationship represents behavior that is factored out of the use case.
- <<include>> behavior is factored out for reuse, not because it is an exception.
- The direction of a <<include>> relationship is to the using use case (unlike <<extend>> relationships).

Use Cases are useful to...

- Determining requirements
 - New use cases often generate new requirements as the system is analyzed and the design takes shape.
- Communicating with clients
 - Their notational simplicity makes use case diagrams a good way for developers to communicate with clients.
- Generating test cases
 - The collection of scenarios for a use case may suggest a suite of test cases for those scenarios.

Where are mistakes?



Learning Objectives

7.1 History of UML

7.2 Systems, Models and Views

7.3 Design Use cases

7.4 Design Class diagrams

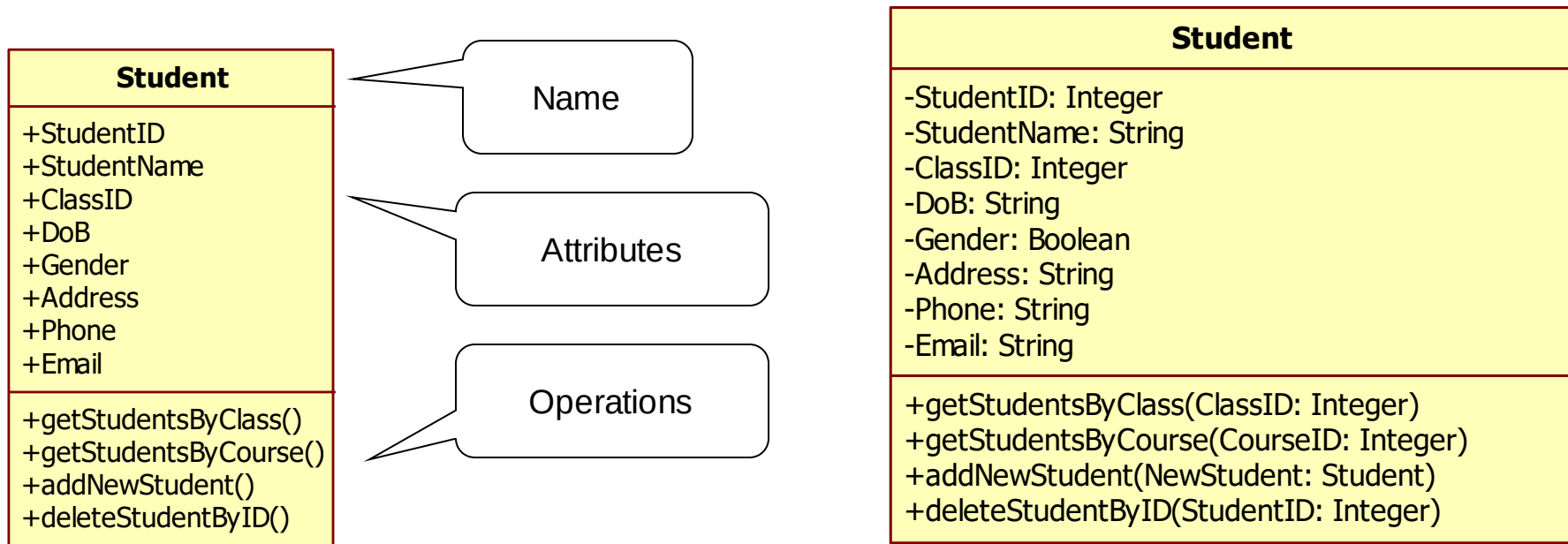
7.5 Design Interaction diagrams

7.6 Design another types of diagrams

Class Diagrams

- Gives an overview of a system by showing its classes and the relationships among them.
 - Class diagrams are static
 - Display what interacts but not what happens when they do interact
- Also shows attributes and operations of each class
- Good way to describe the overall architecture of system components

Classes – Not Just for Code



- A **class** represent a concept
- A class encapsulates state (**attributes**) and behavior (**operations**).
- Each attribute has a **type**.
- Each operation has a **signature**.
- The class name is the only mandatory information.

Class Notation

- A class is a rectangle divided into three parts
 - Class name
 - Class attributes (i.e. data members, variables)
 - Class operations (i.e. methods)
- Modifiers
 - Private: -
 - Public: +
 - Protected: #
 - Static: Underlined (i.e. shared among all members of the class)
- Abstract class: Name in italics

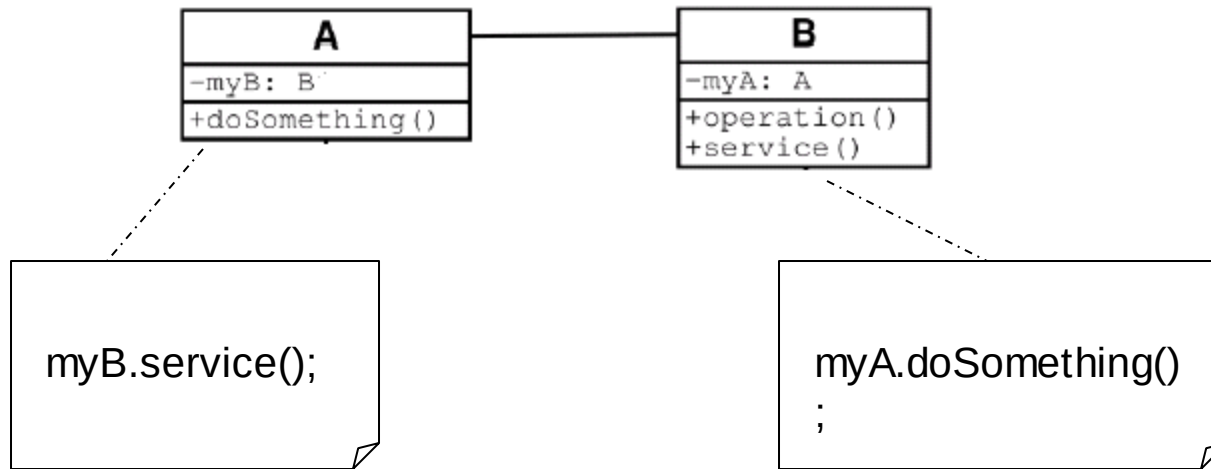
Student
+StudentID +StudentName +ClassID +DoB +Gender +Address +Phone +Email
+getStudentsByClass() +getStudentsByCourse() +addNewStudent() +deleteStudentByID()

Class Diagram: Association Types

- Lines or arrows between classes indicate relationships
 - Association
 - A relationship between instances of two classes, where one class must know about the other to do its work, e.g. client communicates to server
 - indicated by a straight line or arrow
 - Aggregation
 - An association where one class belongs to a collection, e.g. instructor part of Faculty
 - Indicated by an empty diamond on the side of the collection
 - Composition
 - Strong form of Aggregation
 - Lifetime control; components cannot exist without the aggregate
 - Indicated by a solid diamond on the side of the collection
 - Inheritance
 - An inheritance link indicating one class a superclass relationship, e.g. bird is part of mammal
 - Indicated by triangle pointing to superclass

Binary Association

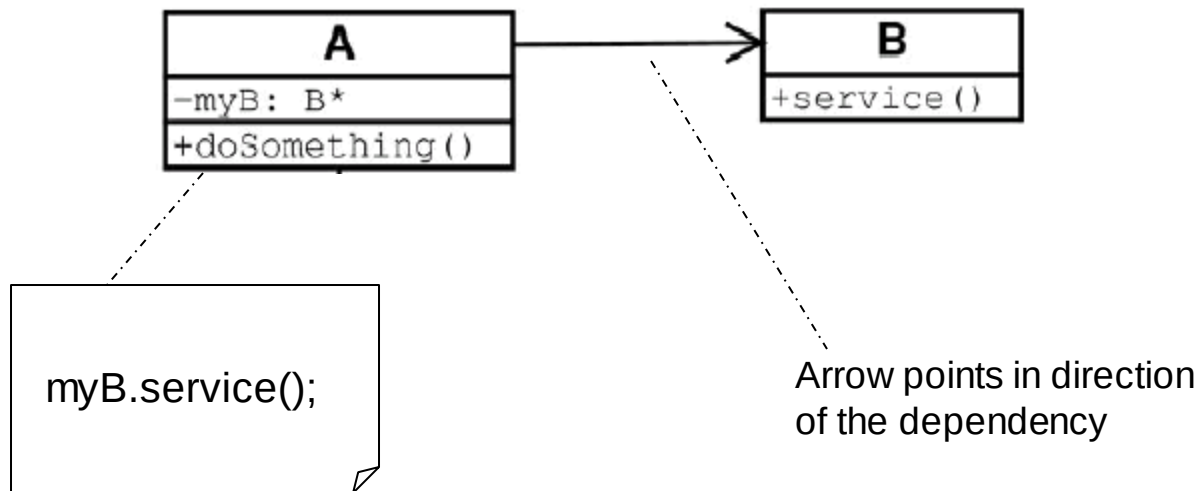
Binary Association: Both entities “Know About” each other



Optionally, may create an Associate Class

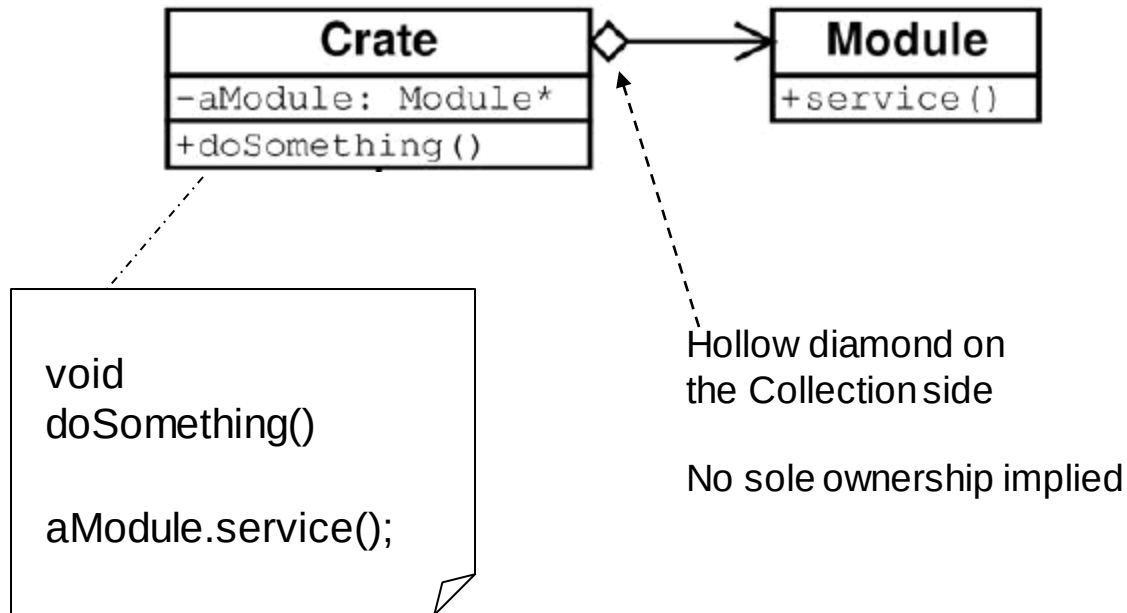
Unary Association

A knows about B, but B knows nothing about A



Aggregation

Aggregation is an association with a “collection-member” relationship

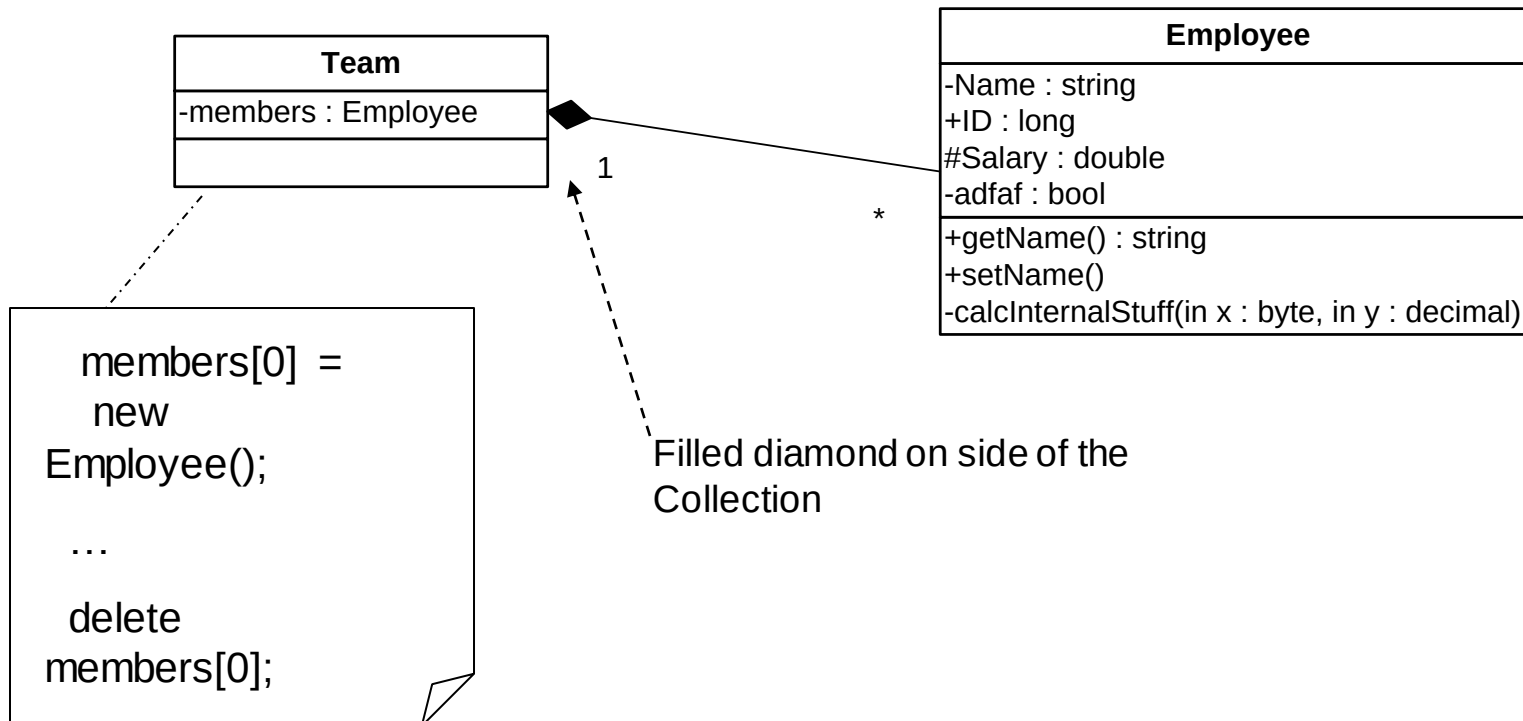


Composition

Composition is Aggregation with:

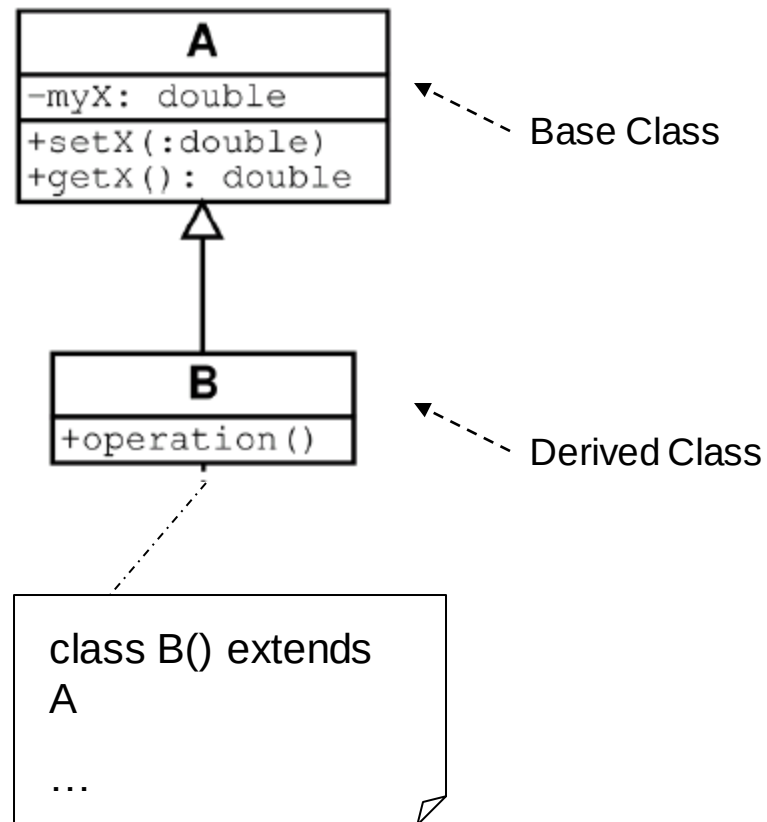
Lifetime Control (owner controls construction, destruction)

Part object may belong to only one whole object



Inheritance

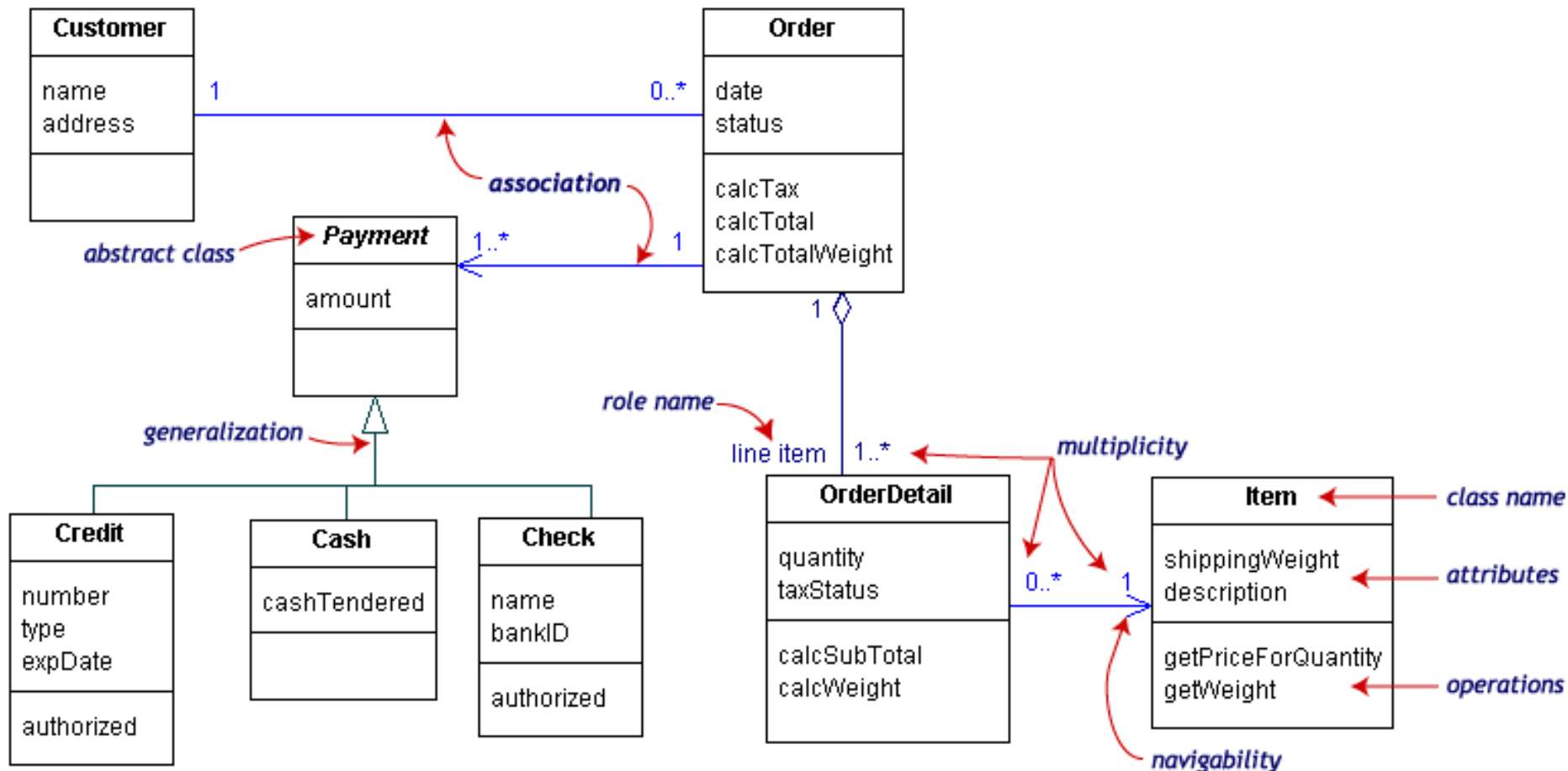
Standard concept of inheritance



UML Multiplicities

Multiplicities	Meaning
0..1	zero or one instance. The notation <i>n</i> . . <i>m</i> indicates <i>n</i> to <i>m</i> instances.
0..* or *	no limit on the number of instances (including none).
1	exactly one instance
1..*	at least one instance

UML Class Example



Learning Objectives

7.1 History of UML

7.2 Systems, Models and Views

7.3 Design Use cases

7.4 Design Class diagrams

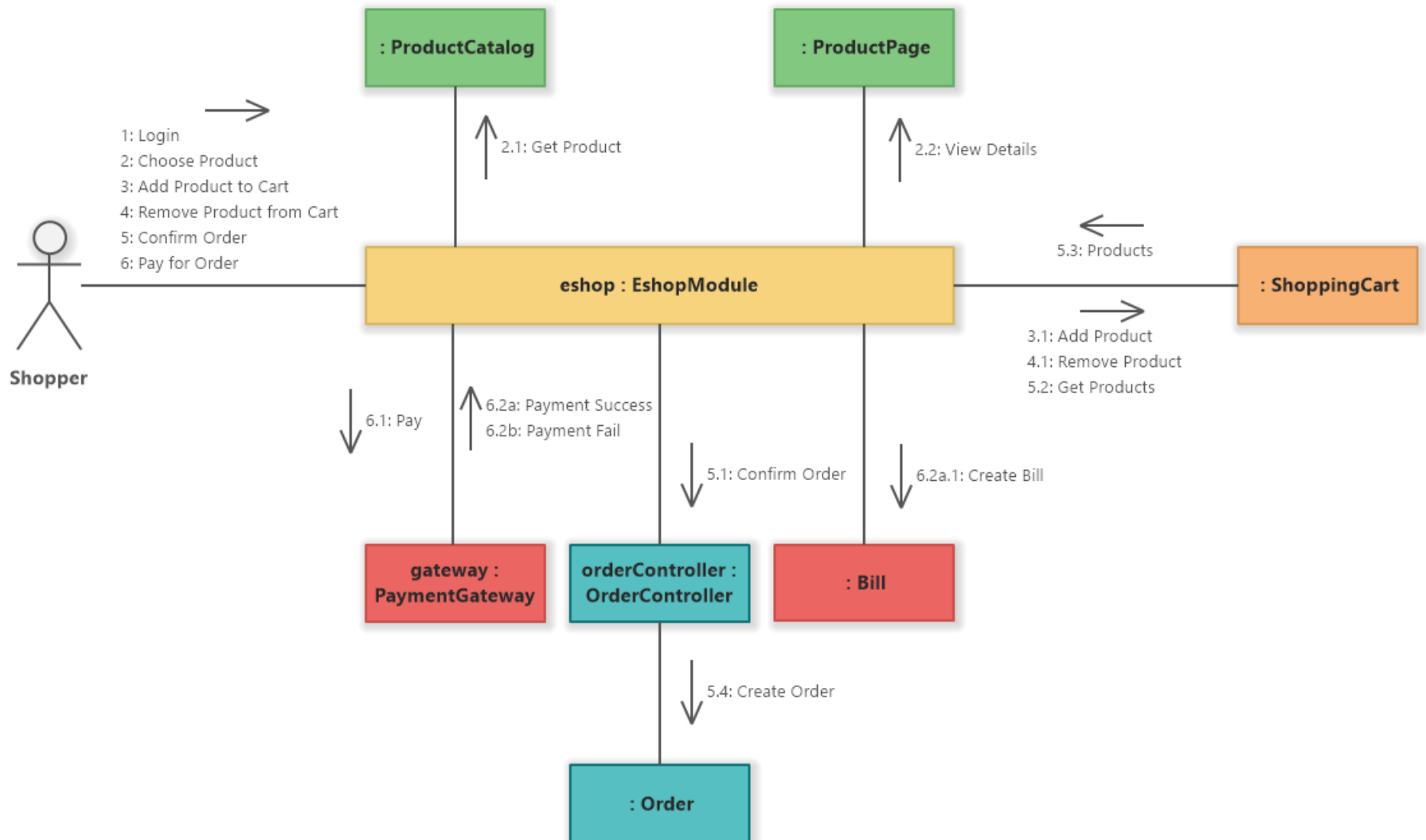
7.5 Design Interaction diagrams

7.6 Design another types of diagrams

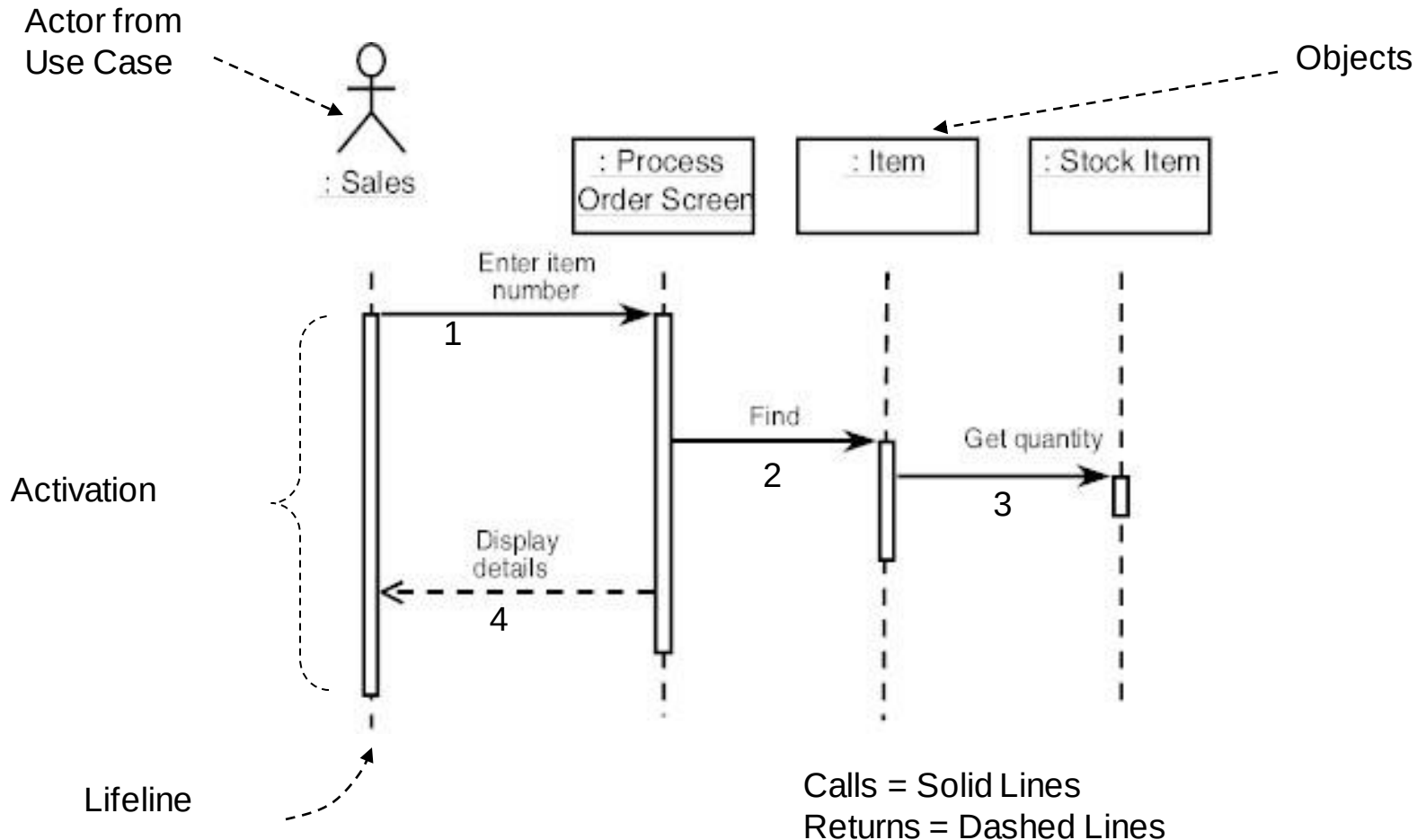
Interaction Diagrams

- Interaction diagrams are dynamic, they describe how objects collaborate
- Communication diagram:
 - Shows the objects along with the messages that travel from one to another
 - Shows the messages the objects send each other
- Sequence diagram:
 - Indicates what messages are sent and when
 - Time progresses from top to bottom
 - Objects involved are listed left to right

Communication Diagram: Example

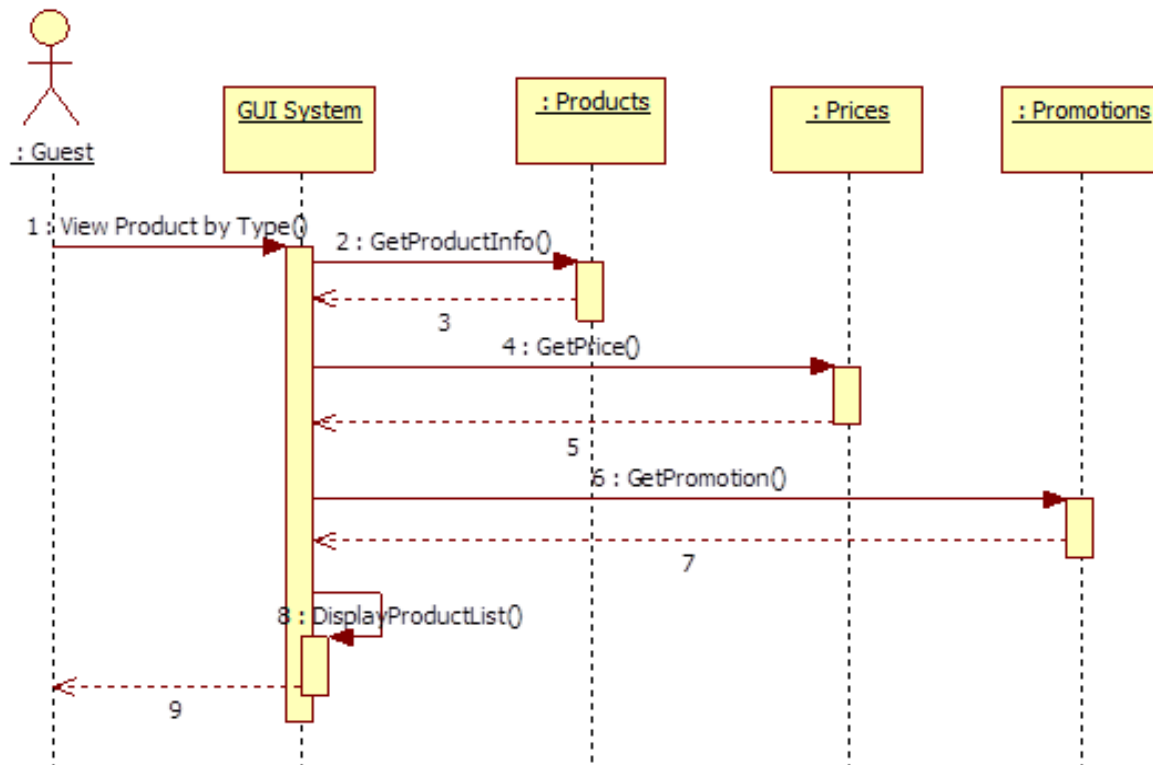


Sequence Diagram: Format

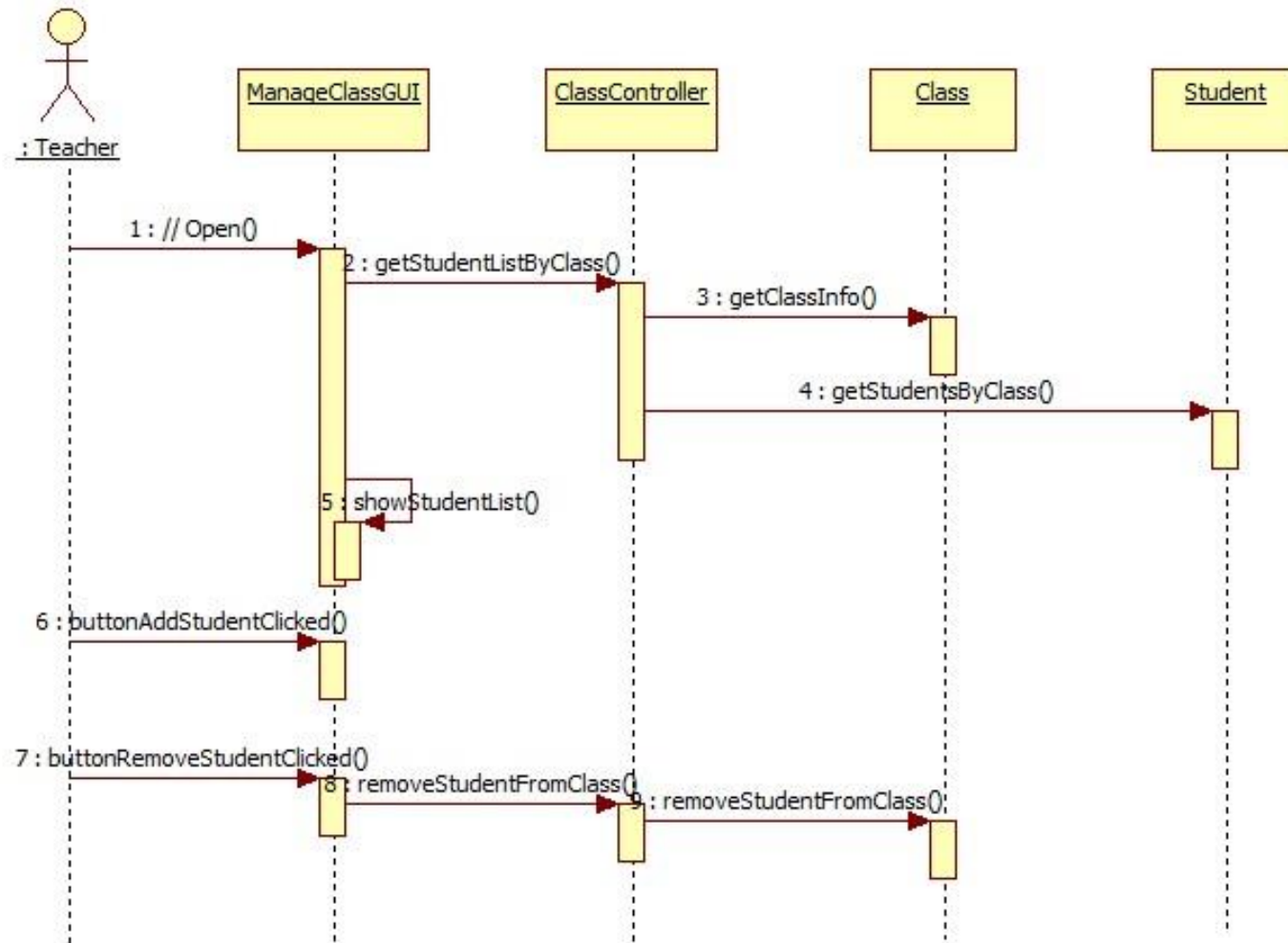


Sequence Diagram: Common Mistakes

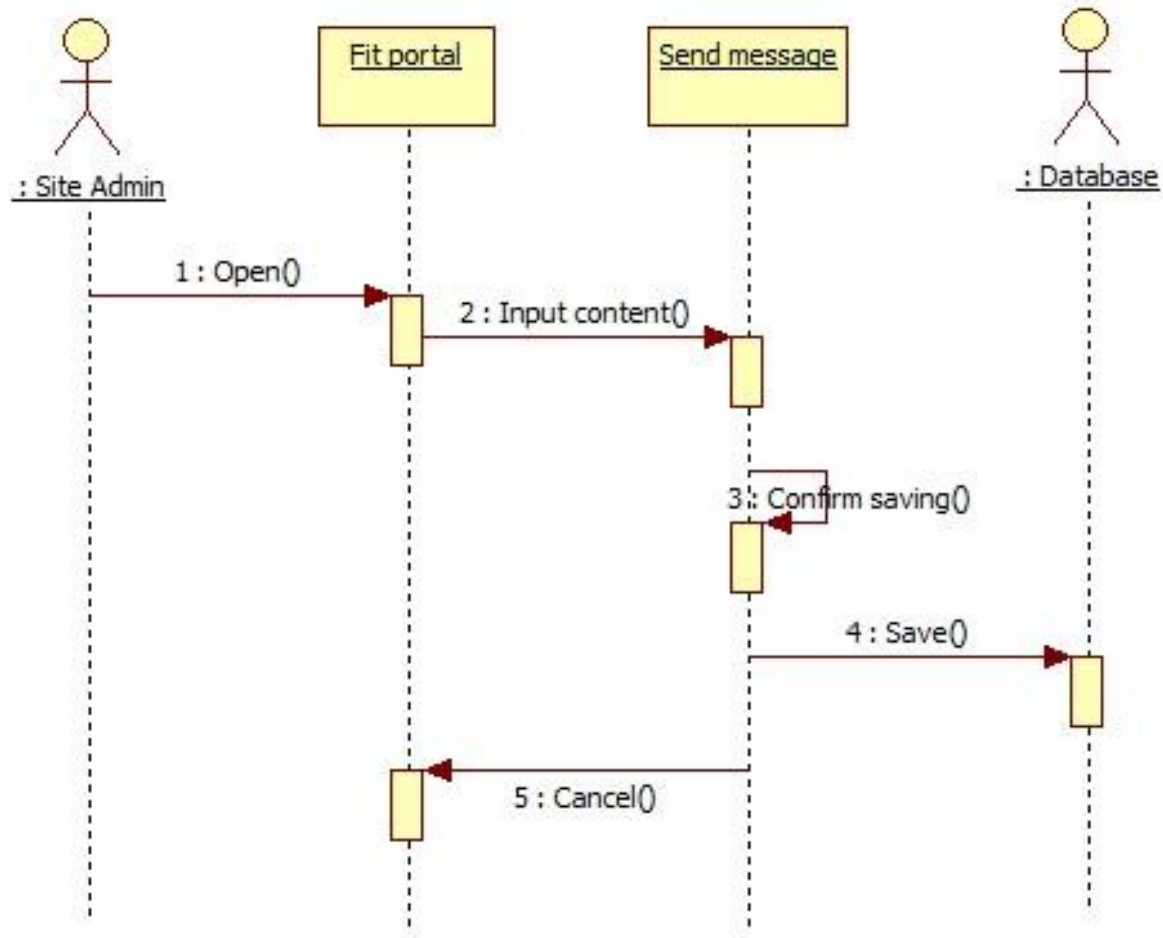
- Meaningless object name such as: Server, Database
- Return message in solid line format
- Too much useless return messages
- Messages from Actor are not exist in target objects



Sequence Diagram: Example



Where are mistakes?



Learning Objectives

7.1 History of UML

7.2 Systems, Models and Views

7.3 Design Use cases

7.4 Design Class diagrams

7.5 Design Interaction diagrams

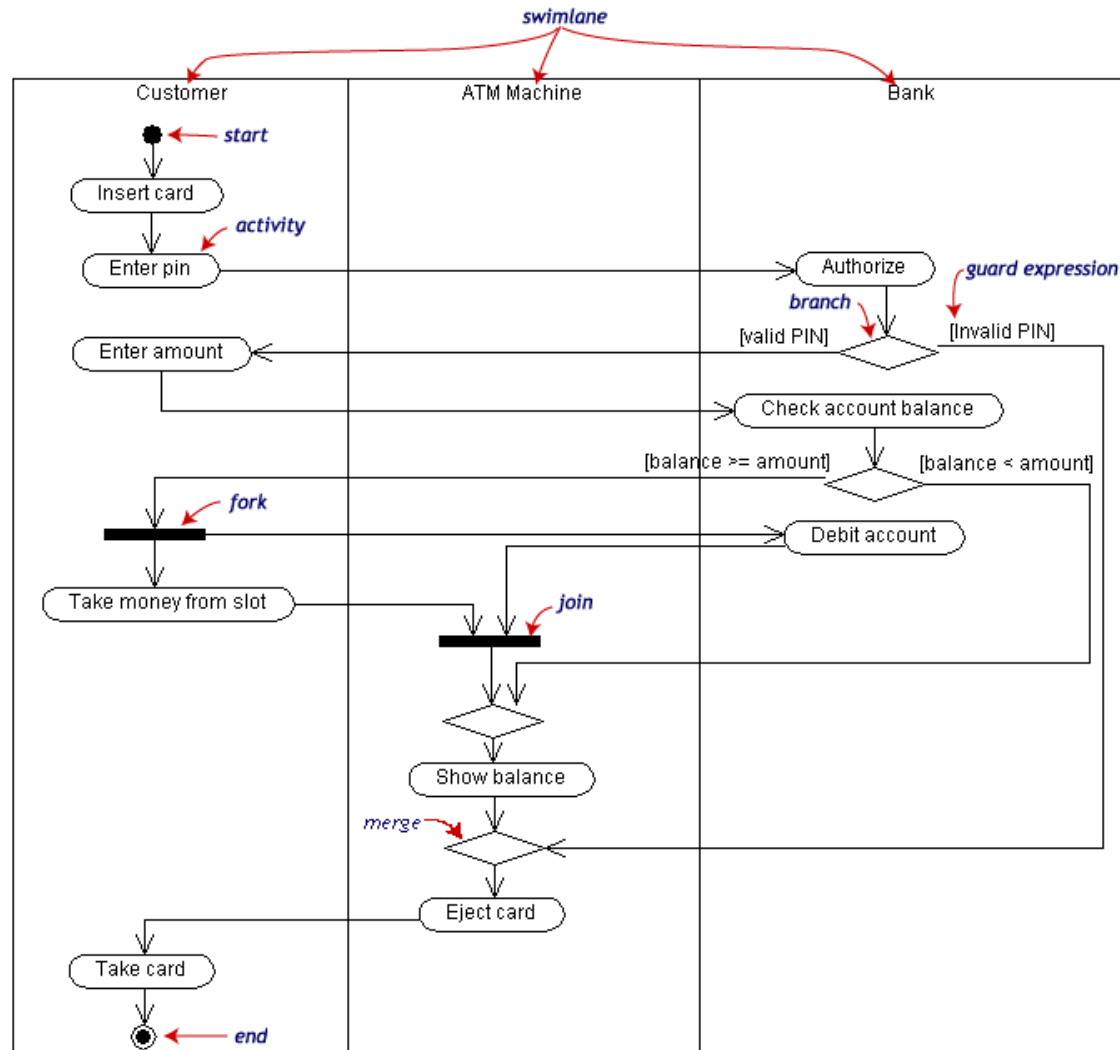
7.6 Design another types of diagrams

Activity Diagrams

- Fancy flowchart
 - Displays the flow of activities involved in a single process
 - States
 - Describe what is being processed
 - Indicated by boxes with rounded corners
 - Swim lanes
 - Indicates which object is responsible for what activity
 - Branch
 - Transition that branch
 - Indicated by a diamond
 - Fork
 - Transition forking into parallel activities
 - Indicated by solid bars
 - Start and End



Activity Diagram Example

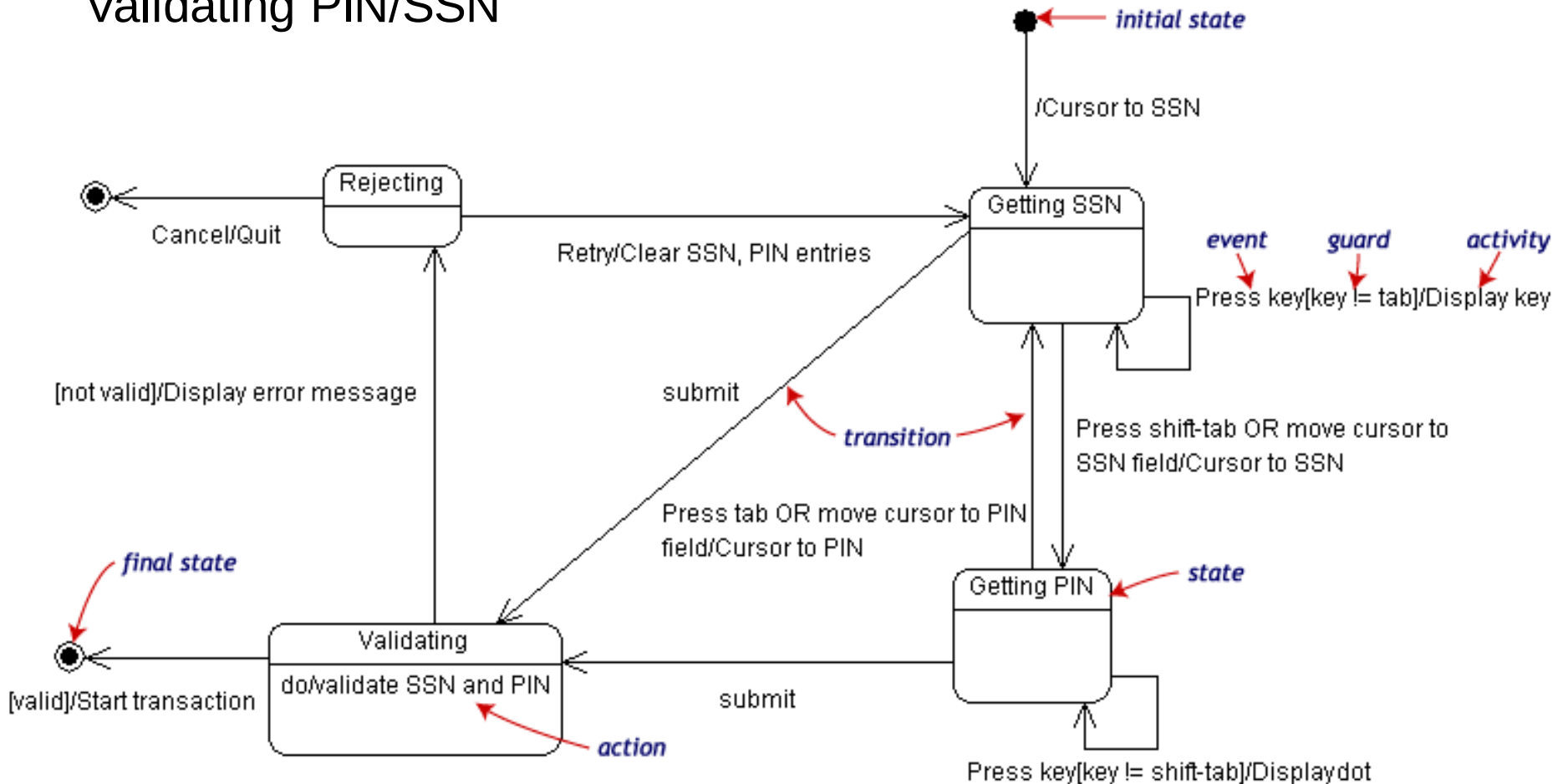


State Transition Diagrams

- Fancy version of a DFA
- Shows the possible states of the object and the transitions that cause a change in state
- Notation
 - States are rounded rectangles
 - Transitions are arrows from one state to another. Events or conditions that trigger transitions are written beside the arrows.
 - Initial and Final States indicated by circles as in the Activity Diagram
 - Final state terminates the action; may have multiple final states

State Transition Example

Validating PIN/SSN

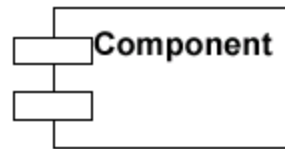


Component Diagrams

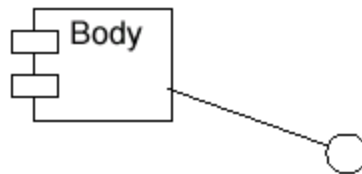
- Shows various components in a system and their dependencies, interfaces
- Explains the structure of a system
- Usually a physical collection of classes
 - Similar to a Package Diagram in that both are used to group elements into logical structures
 - With Component Diagrams all of the model elements are private with a public interface whereas Package diagrams only display public items.

Component Diagram Notation

- Components are shown as rectangles with two tabs at the upper left

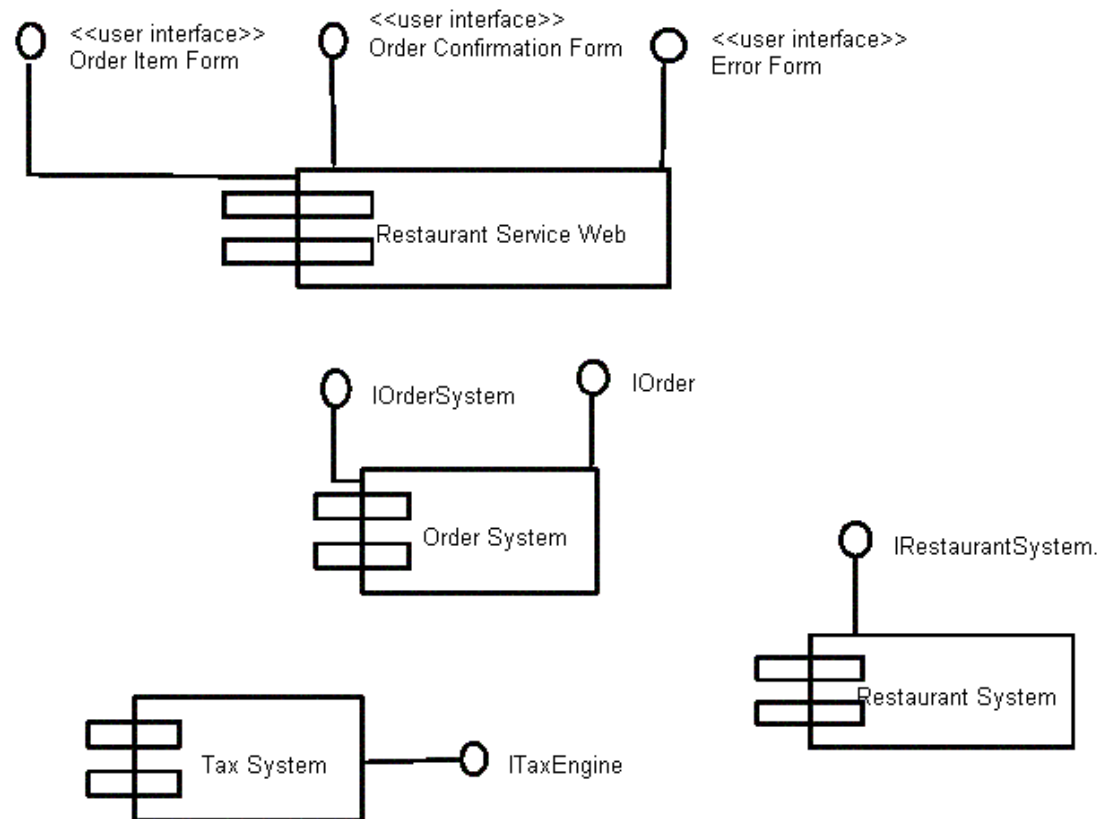


- Dashed arrows indicate dependencies
- Circle and solid line indicates an interface to the component



Component Diagram: Example

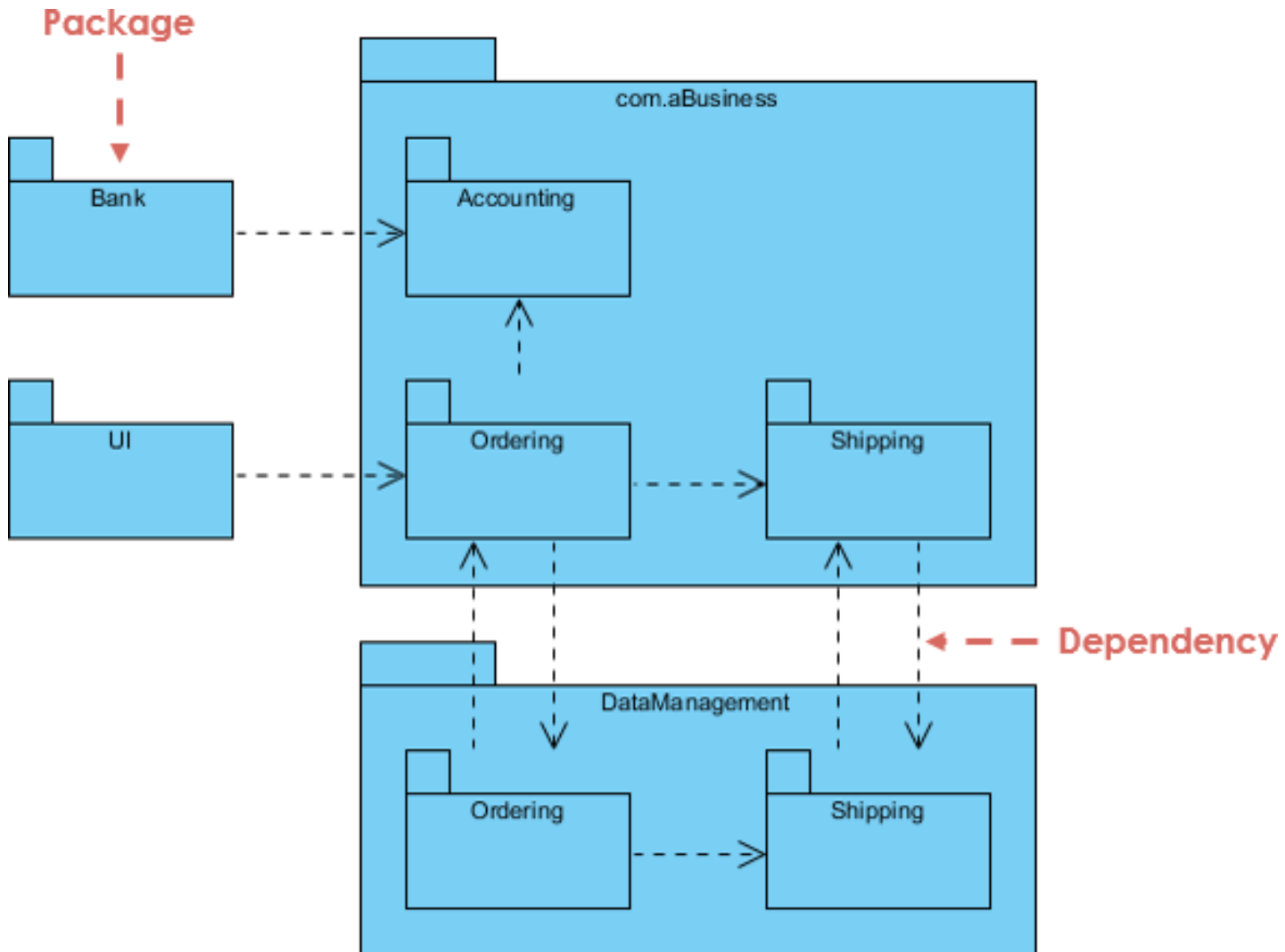
- Graphical depiction of components



Package Diagrams

- To organize complex class diagrams, you can group classes into packages. A package is a collection of logically related UML elements
- Notation
 - Packages appear as rectangles with small tabs at the top.
 - The package name is on the tab or inside the rectangle.
 - The dotted arrows are dependencies. One package depends on another if changes in the other could possibly force changes in the first.
 - Packages are the basic grouping construct with which you may organize UML models to increase their readability

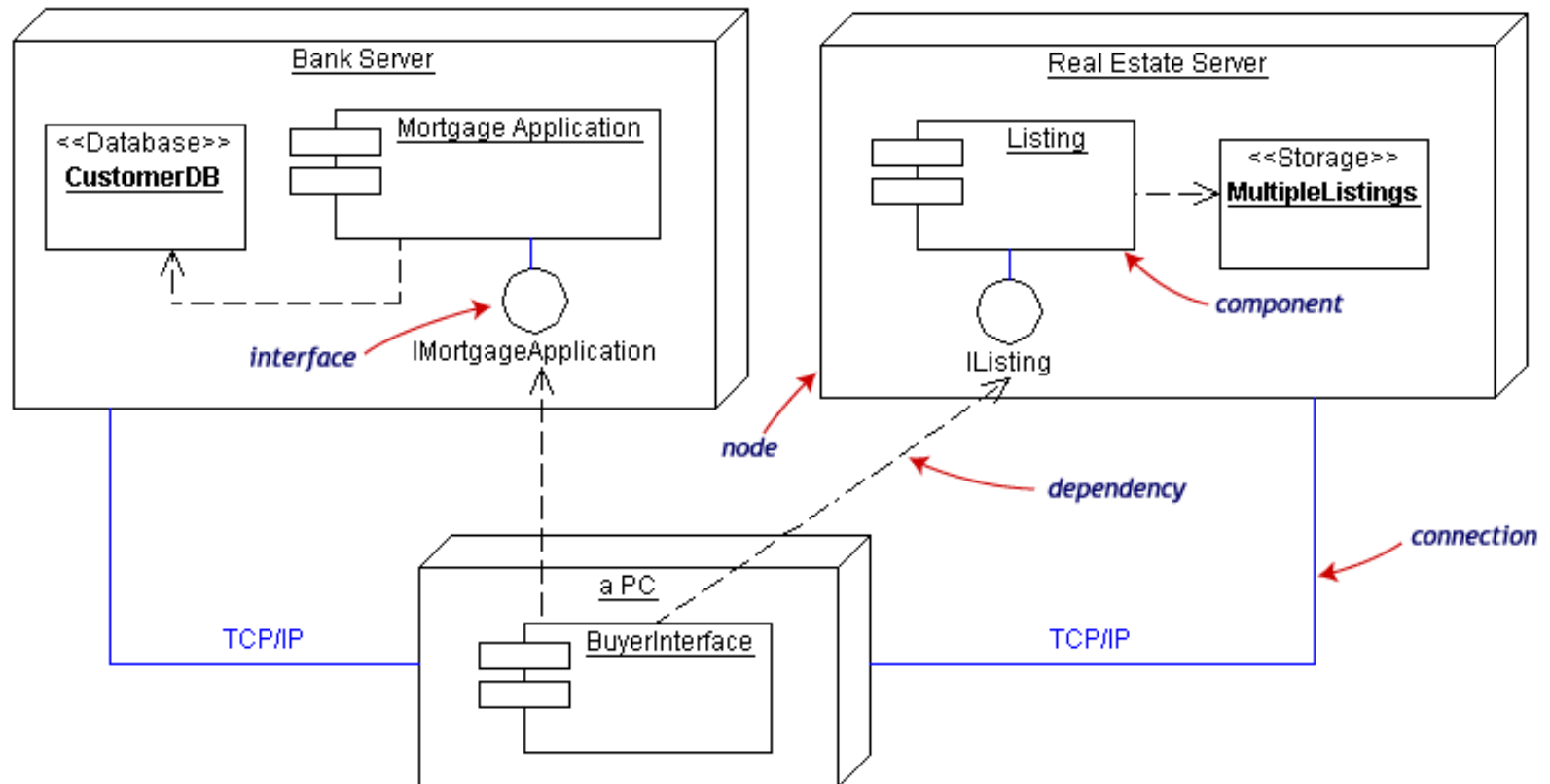
Package Diagrams: Example



Deployment Diagrams

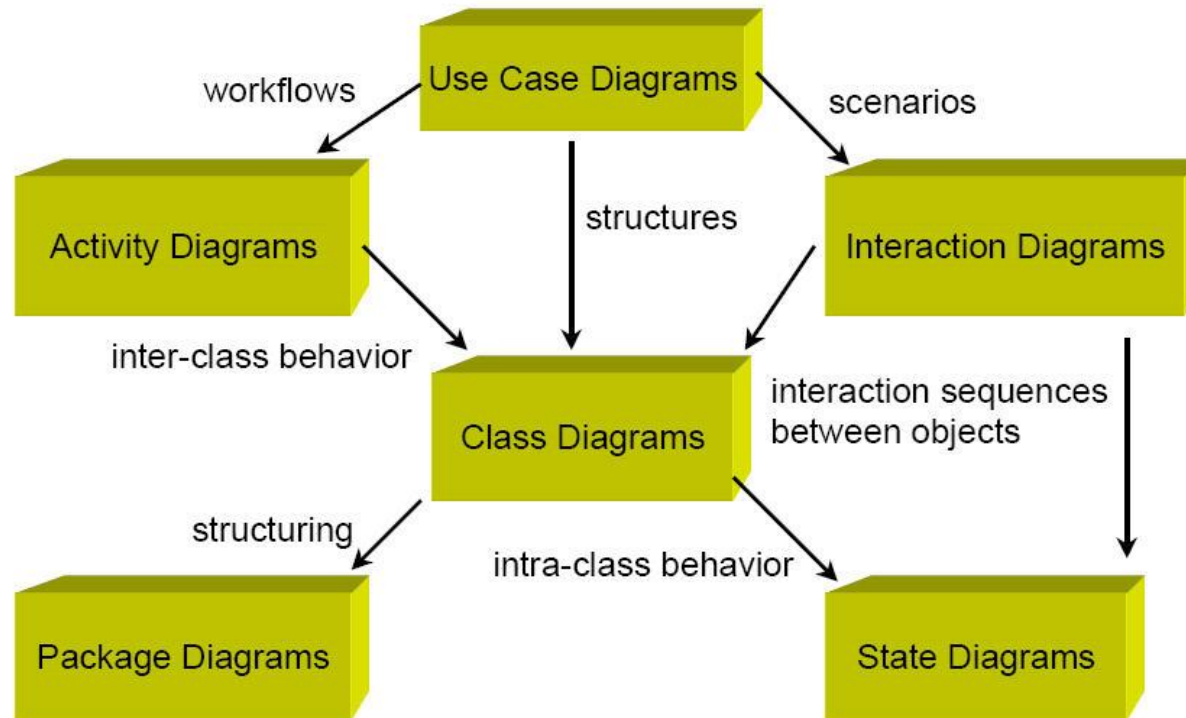
- Shows the physical architecture of the hardware and software of the deployed system
- Nodes
 - Typically contain components or packages
 - Usually some kind of computational unit; e.g. machine or device (physical or logical)
- Physical relationships among software and hardware in a delivered systems
 - Explains how a system interacts with the external environment

Deployment Diagram: Example



Often the Component Diagram is combined with the Deployment

Summary



Activity

1. Draw Fit portal use case diagram
2. Describe the function “Send message” on Fit portal by an Activity diagram
3. Design the function “Send message” by a Sequence diagram

Thanks for your attention